ResearchGate

See discussions, stats, and author profiles for this publication at: https://www.researchgate.net/publication/363469883

Deep Learning in Games to Improve Autonomous Driving

Thesis · January 2022

DOI: 10.13140/RG.2.2.30758.19526

CITATIONS
0

READS
121

1 author:

Rafael Alarcon Borghi

6 PUBLICATIONS **0** CITATIONS

SEE PROFILE

All content following this page was uploaded by Rafael Alarcon Borghi on 11 September 2022.

The user has requested enhancement of the downloaded file.

Deep Learning in Games to Improve Autonomous Driving

Rafael Alarcon Borghi

Dissertation submitted in partial fulfilment of the requirements for the degree of

M.Sc. in Data Analytics

at Dublin Business School

January 2022

Supervisor: Dr Shazia A. Afzal

Declaration

I declare that this Applied Research Project that I have submitted to Dublin Business School for the award of Master of Science in Data Analytics is the result of my investigations, except where otherwise stated, where it is acknowledged by references. Furthermore, this work has not been submitted for any other degree.

Signed: **Rafael Alarcon Borghi**

Student Number: **10563409**

Date: **January 10th, 2022**

Acknowledgements

I would first like to mention that this project could not have been possible without the guidance and support from my supervisor, Dr Shazia A. Afzal. I will be forever thankful for her help.

This research has been partially supported by the Higher Education Authority, of the Irish Government, under the July Stimulus Package.

In addition, I would like to acknowledge my colleagues from the Roblox community for their wonderful collaboration, in special to Kironte, who developed the Roblox Neural Network Library.

Finally, I want to thank my family for their patience and support, in special to my two sons, Fabrizio and Filippo. I hope this research can inspire them in the future.

Abstract

Autonomous Driving is a lively research area in Artificial Intelligence that aims to automatize procedures for self-driving vehicles. Machine Learning has overcome human performance in some simulators with the development of techniques like Deep Reinforcement Learning and Convolutional Neural Networks, as they have been applied with great success to a variety of applications, such as video games. The objective of this work is to demonstrate established Deep Learning techniques used in videogame simulators to show proficiency in Autonomous Driving. In this research, concepts of Autonomous Driving, Video Games and Deep Learning are presented. For that, a simulator is used to enable the use of Machine Learning. Also, a technique related to Deep Learning is evaluated and chosen for the demonstration of conduct vehicles automation in the videogame simulator. Variables related to the agent and the environment (such as distance, steer variation, among others) are taken into consideration.

Keywords: Deep Learning, Neural Network, Artificial Intelligence, Machine Learning, Videogame, Video Game, Roblox, Autonomous Driving, Self-Driving, Car Simulation.

Table of Contents

List of Figures 6
List of Tables 7
List of Important Abbreviations 8
1. Introduction 9
1.1. Deep Learning 9
1.1.1. Artificial Neural Networks 10
1.2. Video Games 14
1.2.1. Driving Simulator 14
1.3. Author's Purpose 15
1.4. Research Question 15
1.5. Aim & Objective 15
1.6. Road Map 16
2. Literature Review 17
2.1. Related Work 17
2.2. Literature Review Summary 19
3. Methodology 21
3.1. Environment Evaluation 23
3.2. Deep Learning Technique Review 24
3.3. Variables Review 25
3.4. Environment Setup 26
3.5. Methodology Summary 29
4. Implementation 30
4.1. Parameter's Setup & Tunning 30
4.2. Results 31
4.2.1. Scenario 1 32
4.2.2. Scenario 2 35
4.3. Implementation Summary 37
5. Discussion 39
5.1. Evaluation 39
5.2. Scope and Limitations 39
5.3. Conclusions 40
5.4. Future Work 40
6. References 42

List of Figures

Figure 1: Multilayer Feedforward Network Simplified Diagram 11

Figure 2: Block Diagram of Supervised Learning (Haykin, 2009)..... 12

Figure 3: Directions of 2 basic signal flows in a DNN: forward propagation of function signals and backpropagation of error signals (Haykin, 2009) 13

Figure 4: Basic Flow Diagram of the Processes 22

Figure 5: Vectors of the 5 Raycasting on the RADAR simulation from the vehicle displayed in neon white 25

Figure 6: Track 01 (Rectangular Circuit) 28

Figure 7: Track 02 ("M" Circuit)..... 29

Figure 8: Screen Capture of Roblox Studio IDE with some Parameters and their Values 30

Figure 9: Screen Capture of Roblox Studio IDE with the DNN Parameters..... 31

Figure 10: Training Car in Track 01..... 33

Figure 11: Testing Car in Track 02 37

List of Tables

Table 1: Summary of Related Work 20
Table 2: Simulators Comparison 24
Table 3: Results for the Test Set on 10% of Total Population (140 Cars) in Track 01 – Part A ... 32
Table 4: Results for the Test Set on 10% of Total Population (140 Cars) in Track 01 – Part B.... 33
Table 5: Results for the Test Set on 10% of Total Population (140 Cars) in Track 02 – Part A ... 34
Table 6: Results for the Test Set on 10% of Total Population (140 Cars) in Track 02 – Part B.... 35
Table 7: Results for the Test Set on 20% of Total Population (50 Cars) in Track 01 – Part A 35
Table 8: Results for the Test Set on 20% of Total Population (50 Cars) in Track 01 – Part B..... 36
Table 9: Results for the Test Set on 20% of Total Population (50 Cars) in Track 02 – Part A 36
Table 10: Results for the Test Set on 20% of Total Population (50 Cars) in Track 02 – Part B.... 37

List of Important Abbreviations

AI – Artificial Intelligence

ANN – Artificial Neural Network

API – Application Programming Interface

CNN – Convolutional Neural Network

DL – Deep Learning

DNN – Deep Neural Network

DQN – Deep Q-Network

DRL – Deep Reinforcement Learning

GPU – Graphics Process Unit

GTA – Grand Theft Auto

IDE – Integrated Development Environment

IL – Imitation Learning

ML – Machine Learning

NN – Neural Network

POMDP – Partially Observable Markov Decision Process

RL – Reinforcement Learning

RNN – Recurrent Neural Network

TORCS – The Open Racing Car Simulator

1. Introduction

Urban locations offer a variety of issues even for the most experienced drivers, and a good approach to mitigating the risks is for humans to train their abilities in simulation software (Păvăloiu et al, 2018) (Tobin et al, 2017). However, this approach is being replaced by "automated digital drivers" which use ML techniques to learn how to drive safely in those environments (Maurer et al, 2016) (Memon et al, 2016). On the other hand, driving is a skill very difficult to measure. The answer for "how good is a driver" is inside a grey area and setting metrics to monitor the driver's behaviour is something challenging, as to choose the right parameters for the algorithm to learn from it (Talpaert et al, 2019). For deploying a project of this nature successfully, the solution needs to provide not only the drivers' functionalities but must have to be very secure for the world. With that, safety is considered a challenging feature for DL projects in autonomous driving (Grigorescu et al, 2020). Fortunately, the testing phase for such systems can be done in a non-real scenario, such as video games, preventing any real fatal casualties and reducing costs (Bratosin et al, 2019) (Almasi et al, 2020) (Tobin et al, 2017).

While video games can be popular and can offer a great amount of data on human-computer interaction, ML algorithms can improve video games about their content and their user experience (Yannakakis and Togelius, 2018). As DL is already largely used by a considerable amount of scene perception devices in autonomous vehicles (Grigorescu et al, 2020), it can be an interesting option to apply the same for autonomous agents in a simulation.

1.1. Deep Learning

Deep Learning (also known as Deep Machine Learning) is a subset of ML techniques based on algorithms that attempt to model high-level abstractions of data using multiple layers of activity vectors as representations, composed of many linear and non-linear transformations (Bengio et al, 2021). It also can be defined as a technique that computes a large configuration of learned functions and/ or concepts for which standard techniques of ML are not addressed (Goodfellow et al, 2016).

It was developed by studies showing how the human brain computes data, having the assumption that each brain layer is learning the features at larger levels of abstraction (Babiker et al, 2019). Some people claim that the first ANN to be trained in this method was designed by Alexey Ivakhnenko in the 1960s with the study of the Group Method of Data Handling (Schmidhuber, 2014).

Some of the purposes of DL lies in the objective of understanding the data correctly and to enables machines to acquire knowledge in a given algorithm with multiple steps (Goodfellow et al, 2016).

DL has many inclinations and a variety of methods have been tried which have yet to be successful. However, it has become the main enabler for Visual Object Recognition, Robotics, Autonomous Driving and Natural Language Interpreters (Grigorescu et al, 2020). Some of the most successful DL methods involve ANN.

1.1.1. Artificial Neural Networks

An Artificial Neural Network (or Neural Network only) is a computational model motivated by an animal's central nervous system (in particular the human brain) which is capable of performing ML operations as well as pattern recognition, mimicking the human brain behaviour, which means gathering new knowledge, making mistakes, and finding out new information. They are inspired by the 1959 biological model proposed by David H. Hubel and Torsten Wiesel who discovered 2 types of cells in the primary visual cortex: simple (to trigger determined features of visual inputs) and complex (which exhibited less spatial variance than the simple ones) (Haykin, 2009). This discovery motivated some modern NN designs ahead (Schmidhuber, 2014).

The ANN is composed of neurons (also called nodes, or processing units), synaptic weights (also known as interneuron connection strengths), and the propagation function. The artificial neurons are inspired by the biological neurons, which receive one or more inputs and deliver one or more outputs. The synaptic weights are used to store the learned knowledge. The propagation function is used to bring the inputs through the neurons. It is composed of the weighted addition of the input values and the bias. The difference from a linear regression model is the activation function, which supplies the non-linearity to the ANN, enabling complex operations with the data (an ANN without it is essentially a linear regression model only). Also referred to as the squashing function, the activation function is used to restrict the proportions of the neuron's output (Haykin, 2009). The neuron performs the weighted sums calculations while the activation function decides if a neuron triggers or not. Types of activation functions are discussed later in this research.

Architecture

Neural architectures are typically grouped in components called layers, with neurons on a given layer to be connected to other neurons in the following layer (Goodfellow et al, 2016). Similar behaviour is noted in the neurons from a given layer. The arrangement of neurons in the layers and the connection pattern between them define the ANN architecture, and the number of neurons in each layer is a significant item in the same (Idrissi et al, 2016).

There are 3 types of layers: input, hidden and output. The input layer is where values are given to the ANN. The hidden layers (also known as intermediate layers) are the place where most of

the processing is performed, through weighted inputs and produce an output through an activation function. Finally, the output layer is where the outcome is presented (Haykin, 2009). If the ANN has no hidden layers, it means it has only the input layer and the output layer, and the value is projected only one way, from the input nodes to the output nodes (also known as computation nodes), it is called Single-Layer Feedforward Networks. If a similar ANN has one or more hidden layers, it can be called Deep Neural Network, or Multilayer Feedforward Network, Deep Feedforward Network or even Multilayer Perceptron (which is also known as a calculation function delineated to a given set of input values to output values). Finally, a Recurrent Neural Network has one or more feedback loops whose feature is not present in the other 2 architectures (Goodfellow et al, 2016), (Haykin, 2009).

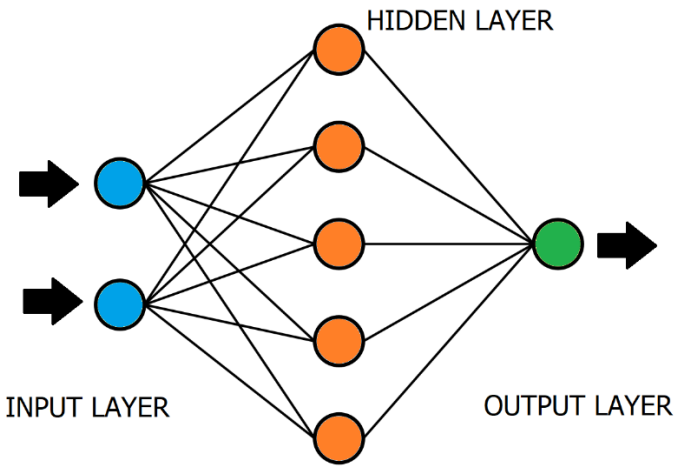

Figure 1: Multilayer Feedforward Network Simplified Diagram

Learning Process

The ANN learns from its iteration with the environment, through a procedure called learning algorithm, which are training rules where the weights of their synapses are adjusted on the received values to achieve a goal. This activity is called the learning process, and can be classified into the following (Haykin, 2009):

- Supervised Learning: when an external agent (such as an instructor) is used to teach the network the desired outcome to the input pattern.
- Unsupervised Learning: when no external agent is indicating the desired outcome to input patterns.
- Reinforcement Learning: when an external critic evaluates the outcome provided by the network.

The learning rate $\eta(n)$ manages the magnitude to change the model on regards the calculated error in every iteration n, when the weights are updated (Haykin, 2009). Smaller learning rates

cause the training process to be longer while a higher value can incur a rise in the average loss (Bengio, 2012).

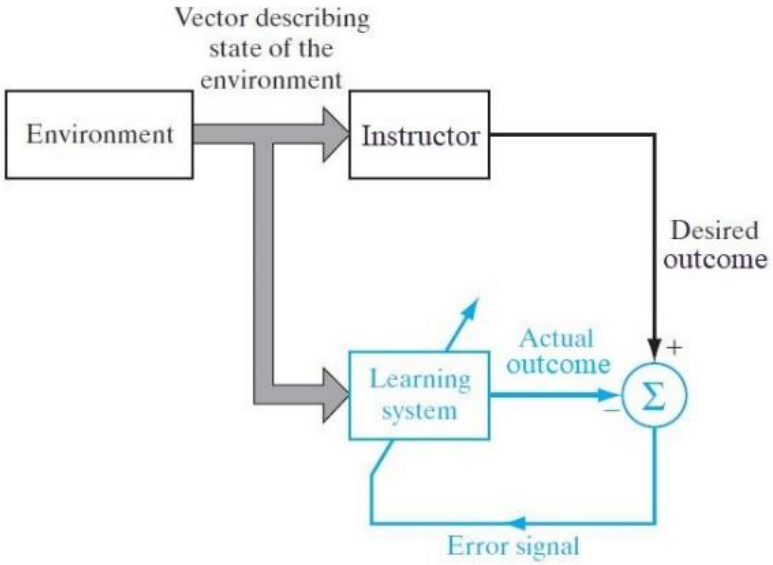

Figure 2: Block Diagram of Supervised Learning (Haykin, 2009)

One of the objectives of the learning process is to reduce error. NNs learn through optimization of their weights (and biases) (Parr and Howard, 2018). For training a DNN it is necessary to select the optimizer and the cost function (also known as loss function, or error function). Optimizer is the algorithm that, given a set of parameters, returns a value by minimizing (or maximizing) some function f(x) by changing x. The optimizer function tries to minimize or maximize is known as the objective function or criterion (meaning that it is searching for a solution having the lowest or highest score respectively). It is called the cost function the objective function for minimizing operations. The cost function returns an outcome showing how well the ANN models the data. Higher values from the cost function mean that the algorithm is not adherent to the objective (Hennig and Kutlukaya, 2007). As it is the function for measuring the accuracy on the performance error of an ANN, they are similar to the ones used in other parametric models (such as linear models), as they define a distribution $p(y|x; \theta)$ (Goodfellow et al, 2016). Types of cost function and optimizer algorithm are discussed later in this research.

An accepted technique for training ANNs is backpropagation. It was introduced in 1985 by Hinton, Rumelhart, and Williams and has 2 phases (Haykin, 2009):

- Forward propagation: the weights are set, and the input values are passed on through the network until they reach the output nodes. Modifications are restricted to the activation possibilities and nodes outputs.

- Backward propagation: an error signal is created by comparing the output with the expected value. Then, the error signal is passed on through the network, but in the inverse direction, in which several changes are made on the weights.

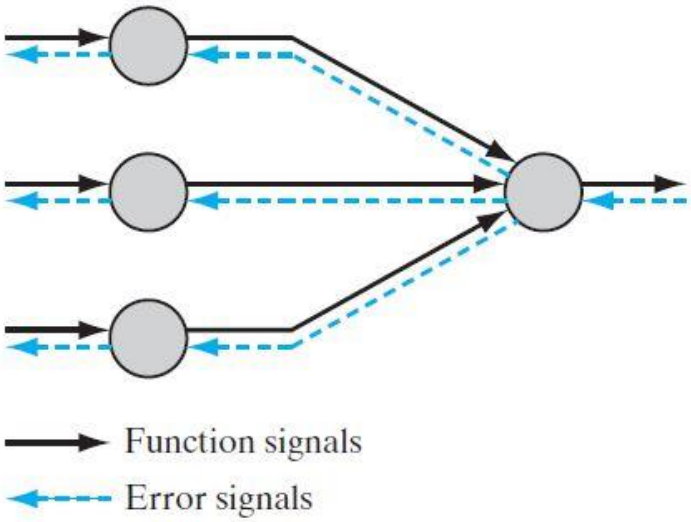

Figure 3: Directions of 2 basic signal flows in a DNN: forward propagation of function signals and backpropagation of error signals (Haykin, 2009)

This backward flow of the error information allows for efficient computation of the gradient at each layer. Calculating an analytical expression for the gradient is straightforward, but numerically evaluating such an expression can be computationally expensive, in special for the hidden layers (Goodfellow et al, 2016).

Finally, overfitting (or overtraining) is when a model has a good performance for the training data but a poor performance in the test/ unknown data. When it happens, the ANN is not able to discover patterns on both input and output standards (Haykin, 2009). There are a few ways to handle overfitting, from modifying some of the hyperparameters or applying some techniques such as early stopping (decide how long to train for a piece of information on a given hyperparameter by monitoring the out-of-sample error on the training process) (Bengio, 2012).

Hyperparameter

In ML, hyperparameters are settings to manage the actions of the ANN algorithm (Goodfellow et al, 2016). There are several hyperparameters needed to be specified such as:

- The number of input values.
- The number of output values.
- The number of hidden layers.
- The number of neurons in a given hidden layer.
- The optimizer algorithm.
- The activation function of a given hidden layer or output layer.

- The learning rate.

If the number of hyperparameters in a model is fixed or variable, it indicates the same as parametric or non-parametric respectively (Hennig and Kutlukaya, 2007), (Goodfellow et al, 2016).

While some hyperparameter's values can be known in advance, empirical tests must be performed for others, as several attempts may be necessary to reach the optimum.

1.2. Video Games

Video Game is an electronic game made mainly for entertainment purposes in which the player interacts through peripherals connected to the machine, such as controls (joysticks) and/ or other devices (e.g.: keyboard, mouse) with images displayed on a video screen (normally a television, a built-in screen from a handheld device or a computer monitor). There are many genres of videogames: first-person shooters (FPS), real-time strategy (RTS), racing simulators, role-playing games (RPG), and card games are just some examples.

There are few discussions on when video games first came, but some authors like Wolf and Crookall point out it was in the 1950s when the video gaming age began (Wilkinson, 2016).

Some video games provide a realistic experience for gamers but are difficult to be used for anything other than entertainment as it would require some illegal methods to achieve the game resources (Rong et al, 2020). To sort out this situation, some games provide their development platform, while others are designed to that end, such as the Serious Games.

Serious Games

Serious Games are video games and simulation approaches and/ or technologies whose primary purpose is not the entertainment itself, which might have different applications, such as marketing, education, training, among others (Wilkinson, 2016). Moreover, the user's experience in the Serious Games can enhance their knowledge on a given area, which might change the player's views and behaviour on the same (Vidakis et al, 2019). Data is generated during the gameplay and can be captured by an API which is a connection software to act as an intermediary that enables two or more applications to communicate between them. These software components are very useful for many use cases, such as deploying ML models into an agent inside a game as the API can control the variables of the environment simulation (Rong et al, 2020).

1.2.1. Driving Simulator

Driving simulators are electronic applications used for many purposes such as entertainment and training. The goal is to place the player in a virtual environment that replaces reality. Earlier than the video games age, acquiring knowledge-based simulation was a mandatory feature given the scenario of real-life problems, and the rules in the simulator can mirror reality in many

aspects (Wilkinson, 2016). As the car simulation software is oriented by the interactions of human players, it tends to be more focused on educational aspects, such as respecting road rules, avoiding accidents, and others (Bratosin et al, 2019), but there are many examples of entertainment options of that genre. Unlike the GTA video game (where human players must perform actions against the law, and the goal is not to be caught by the police) (Encyclopaedia Britannica, 2017), driving simulators main purpose in most cases is to create an awareness culture for the players to follow codes of good conduct for drivers.

Decoding Autonomous Driving problems was in place earlier than ML techniques became popular (Wang et al, 2018). To have an experience more faithful to reality, an important role is played by ML algorithms in the simulation (Bratosin et al, 2019).

1.3. Author's Purpose

Helping people to improve their lives is a very noble task. Automatizing vehicle driving (or at least contributing in some way to this end) is something that should be a primary goal for mankind. Not just figuring out which technique or algorithm is the best or the most accurate one but making those new features available to most people as well.

Also, bringing machine learning knowledge to the games' universe can prospect future developers, as many young people can be inspired by a project like this one. Some game platforms have most of their users and developers under the age of 18, and a project inside their world might inspire a new generation of gamers and scientists.

Finally, it is very gratifying to be able to work on a project that unifies ML, video games and autonomous vehicle driving.

1.4. Research Question

The main question of this research is: How does the application of DL in games improve autonomous driving?

1.5. Aim & Objective

The present work suggests the application of DL techniques to show the proficiency of an autonomous vehicle driver (agent) in a video game simulation (environment). With that, the work aims to address how the algorithm can learn from interaction with the platform. The goal is not necessarily making the driver be a faster one, but to demonstrate the ability to conduct a vehicle with aptitude, which means avoiding obstacles like sidewalks and making curves accurately.

The objectives are the following:

- Explore techniques of DL related to autonomous driving.
- Decide on a suitable technique to query on this work.

- Perform experiments to demonstrate the vehicle conduct with the aptitude to avoid obstacles.
- Validate and test the accuracy of the developed model.

1.6. Road Map

The strategic plan for the research project is the following:

- Introduction: includes the background, Deep Learning, Video Games, Author's Purpose, Research Question and Aim & Objective sections.
- Literature Review: a study comparing related work on the topic of this dissertation.
- Methodology: describe the methods used to evaluate and select the DL technique and the variables, and describe the features deployed to build the simulator.
- Implementation: investigation of the parameters and the analysis of the results.
- Discussion: evaluation of the findings, the conclusion of the project and some information of scope and limitations. Also, present proposals for future work.

2. Literature Review

This chapter provides an overview of current knowledge in DL techniques applied to autonomous driving using videogame simulation, enabling recognizing relevant methods, theories, and issues in the existing studies. It focuses on previous research identified by the author to address the project question. At the end of this section, the studies are analysed, explaining the advantages and limitations of their techniques.

2.1. Related Work

El Sallab et al (2017) presented a framework with the use of DRL techniques for self-driving vehicles, as it has advantages among other learning approaches in ML due to the solid interactions with the simulation domain. The authors deployed an RNN in TORCS (an open-source 3D car simulator video game which is developed in C/ C++) to address the POMDP scenario, and a CNN to process images important to the driving operation, with the module Simulated Car Racing to control the variables related to the vehicle. They concluded that the model had a quicker and better execution by removing the replay memory trick feature in the DQN (for discrete actions), but hard-coding the car actions would make some bad performance in the curves, which did not happen with Deep Deterministic Actor-Critic (for continuous actions). In the end, they noticed that both approaches were trying to stay in their lane and keeping some speed even with a decreasing supervision level rate which started with a value of 0.5.

Martinez II et al (2017) research from Princeton University used the well-known GTA V video game as the virtual environment to train a CNN for processing images and calculating the distance between the vehicle and other objects, allowing the algorithm to take proper actions. To do that, the authors used the AlexNet architecture with Keras and TensorFlow as the backend of the model with the Rage Plugin Hook mod to allow the setup of the simulations once the application was not made for scientific purposes. They also used the MSE as the cost function to evaluate the performance of the variables, in special the ones linked with lane markings distances. After running the model for more than 36 hours in a machine equipped with a 12GB Nvidia Tesla K40 GPU, the authors concluded that GTA V enabled the proper use of DL techniques for autonomous driving study despite the limitations of the video game.

Similar work has been done by Chaudhari et al (2018) who also deployed a CNN in the GTA V video game. Having a machine equipped with a 16 GB DDR4 RAM and a 6GB Nvidia GeForce GTX 1060 GPU, the authors used around 100 Gigabytes of driving data from the simulation into the Inception V3 model from Keras module in python, and they used DeepGTAV mod to deal with the characteristics of the video game environment. For the DNN configuration, the authors deployed a Stochastic Gradient Descent optimizer with Nesterov Momentum, and each epoch

took more than 1 hour to finish. At their conclusions, they found that the reason the results were good was related to balancing the data and they recommended using LIDAR and RADAR data to improve the model. Also, they noticed that the learning rate was a key factor for the training phase which the optimized value being 0.001.

In another research with CNN using AlexNet architecture in the GTA V, Kim et al (2019) set up a model using Python Image Library with a Windows API to sort out issues with regards to the response time. Unlikely the studies from Martinez II et al (2017) and Chaudhari et al (2018), the authors deployed an extra step, testing the trained model in the real world with a Raspberry Pie control-based toy car. In their remarks, they concluded that this model learned how to drive in both the video game and in the real simulation observing multiple conditions such as day and night, but also had many issues dealing with training data due to the variety of available vehicles.

Cao et al (2020) offered to combine RL and IL into Hierarchical Reinforcement and Imitation Learning (H-REIL) to handle situations where near-accident driving is frequent by evaluating distinct driving behaviours. The IL piece would understand the low-level policies as each of them refers to a different driving mode (timid and aggressive), and the RL part, modelled as a POMDP, would learn the high-level policy related to the main decisions. To simulate all the proposed scenarios (5 in total), the authors built an environment with the agent (ego car) and a rogue vehicle (ado car) in 2 different car driving simulators: CARLA (a well-known open-source 3D software designed for research in self-driving vehicles) developed by Dosovitskiy et al (2017), and CARLO (a 2D autonomous driving application developed by the researchers in python which controls physics made for low budget environments). For both simulators, they used car location and velocity as the main inputs for the DNN like a RADAR suggested by Chaudhari et al (2018), and throttle/ brake and steering for the main outputs. However, they also used front-view images as input with Proximal Policy Optimization for the CARLA simulator, setting up then a CNN for it. For testing purposes, the researcher team assembled both a single IL and H-REIL models to try both driving modes. At their interpretations, the authors mentioned that H-REIL performed better in both applications compared to tests with single IL, swapping between timid and aggressive behaviours for specific situations.

Moving away from imaging processing and the CNN approach, Fathi and Izadi (2020) proposed a DNN trained with variables such as speed and distances from obstacles, similar as used by Cao et al (2020) in their CARLO simulator. The car had 5 sensors: in the front, on each side (left and right) and another 2 between each side and the front (front & left and front & right). The last 2 sensors were only applied later in their research when they realized that the 3 other sensors only were not good enough to provide data for driving properly. For training the DNN, the authors used backpropagation with Mean Square Error as the cost function and Leaky ReLU as

the activation function, deploying a risk value formula in the sensor variables for calculating how likely the car would crash with an obstacle. In the simulation environment, they employed pathfinding and waypoint navigation in Unity with the Physics package, which enabled ray casting to be used in the sensors for training purposes. The authors concluded that the curves in the racing circuit played an important role to enforce diversification in the training data and that the training agent should drive with less error as possible to ensure the correct learning procedure would be propagated into the test scenario.

Having employed TORCS with the Epic controller, Albelihi (2014) developed the Gazelle, an adaptive algorithm for racing simulation to boost the response against rival cars and to improve the overall performance in the track. Two approaches were adopted in the project: the procedural method, which is made by simple coding to perform driving actions, and a DNN model. Alike Cao et al (2020) and Fathi and Izadi (2020), distances from objects were also used as inputs for both proposals, but angles of the road direction to verify the last state and the current state were added, which the steering direction as the output. The DNN was configured with 2 hidden layers, Tanh as the activation function and the backpropagation for learning intention. After tests in 3 different race circuits, the authors concluded that despite the procedural method taking less time to complete the laps compared to the ML approach, the DNN achieved the goal to outperform the EPIC driver.

Later, Vrajitoru (2018) used the inputs from Albelihi (2014) to deploy a DNN for improving the pathfinding algorithm of the self-driving car in TORCS. The author addressed Meep and Gazelle driver modules for the experiment and mapped the curvature of the tracks using the distance inputs from the car. For the study, three DNNs with backpropagation were set up: the first one had only the curvature as the input with 1 hidden layer of 3 neurons; the second network had also the curvature as the input and two hidden layers of 5 and 3 neurons respectively; the last DNN setup had the same setup of the second one with an additional input of the track edge distance. All three were tested in 4 racetracks of the simulation with cross-validation, and the best performance observed was from the DNN with a single input and two hidden layers after a cautious selection of parameters.

2.2. Literature Review Summary

Having the mentioned studies analysed, it is clear that there are plenty of options to train an autonomous driving agent, from simple ANNs with no hidden layers to more complex CNNs.

They can be summarized into the following table:

Author	Driving Simulator	ML Technique
Cao et al (2020)	CARLA	DNN and CNN
Cao et al (2020)	CARLO	DNN
Martinez II et al (2017)	GTA V	CNN
Chaudhari et al (2018)	GTA V	CNN
Kim et al (2019)	GTA V	H-REIL with CNN
El Sallab et al (2017)	TORCS	DQN with CNN
Albelihi (2014)	TORCS	DNN
Vrajitoru (2018)	TORCS	DNN
Fathi and Izadi (2020)	Unity	DNN

Table 1: Summary of Related Work

It is comprehensible that using CNN to process images is a best practice of ML in autonomous driving as it enables the car to "see" the environment and act just like a human driver (or even better). However, it is recommended to use robust hardware with dedicated GPU in this approach as it deals with a heavy process of video/ images. On the other hand, using a pure DNN algorithm can have a lot of limitations since the car is not recognizing an image but, instead, it is just receiving inputs like the distance of some sort of things (such as buildings, fences, road lanes, etc.). Nevertheless, it can be deployed in almost any machine nowadays as this method doesn't require a very fancy device. For this research, any of them is sufficient since both techniques can make the autonomous driving agent learn how to conduct a vehicle. Finally, the 2 mentioned approaches related to RL (DQN and H-REIL) are way more complex than the previously explained techniques and might not be available in some applications (requiring external customised libraries).

3. Methodology

In this section, the methods used to do the research are discussed. It clarifies the steps taken to achieve the results, providing the details on the path to reach outcomes related to the research question. Some of the procedures were inspired by the studies discussed in the previous section. Methodology is a set of methods used to collect and evaluate data throughout the research process. Its purpose is to provide researchers with how to check the credibility, validity, and reliability of a chosen approach.

The research method is through the following steps:

1. Select and evaluate a driving simulation video game that enables the deployment of Machine Learning algorithms. Licensing/ costs, coding language, and how user-friendly the application is being taken into consideration. Computational limitations such as hardware and environment are considered as well.
2. Review and select an adequate approach of the DL techniques to apply to the use case among the most current and accurate ones. The technique must be available in the simulation software through its native components or external module/ API. Also, it must be related to some work already mentioned in the Literature Review section. Techniques not suitable for the software are discarded, and if no DL techniques could be used in the driving simulation video game, the same is discarded and another evaluation in the previous step is required.
3. Review variables and information. Once available, the information of the simulation software is evaluated on regards the feasibility to perform tasks related to the selected DL technique. Methods on how to obtain the information are taken into consideration as well.
4. Setup and train the algorithm for being able to correct conduct vehicles in the selected video game simulator. In this step, a policy is defined to train the AI agent. After the training process, the DL algorithm tries to learn the driving process and apply the same to the environment. This process might be repeated several times to refine the outcome.
5. Analyse the results. With the outcome presented by the actions of the AI agent, data is analysed, and a conclusion is made whether the algorithm achieved the proposed goals of the present work or not.

The first 4 steps are described in this section while the last one is informed in future sections.

The following picture draws the processes of the simulation:

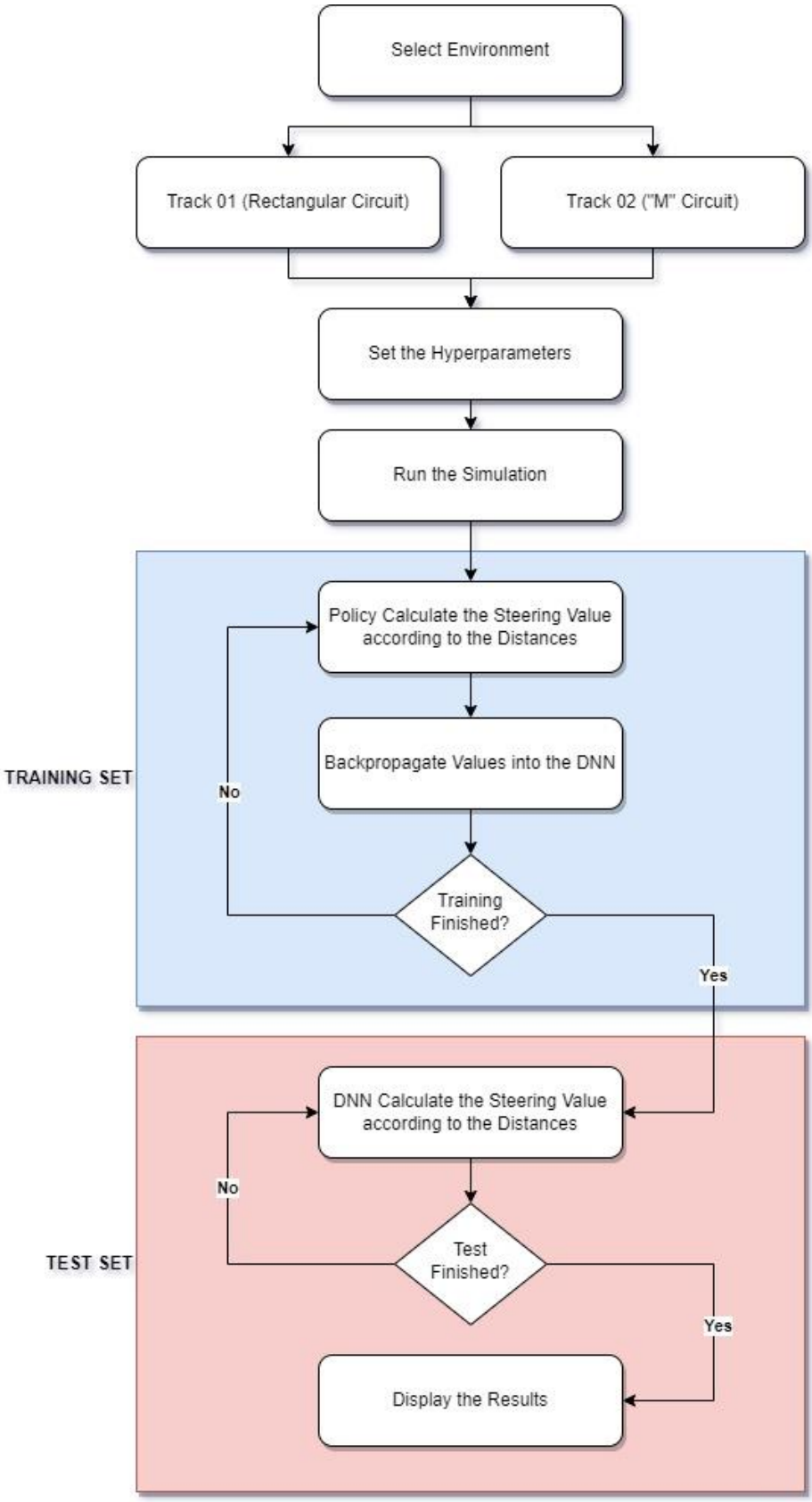

Figure 4: Basic Flow Diagram of the Processes

3.1. Environment Evaluation

To evaluate the suitability of video game simulators using DL techniques, a few items have to be taken into consideration. Having hardware as a possible limitation (the currently available machine for the research does not have a dedicated GPU), there is a tendency to avoid approaches involving imaging processing like the ones using CNN. Therefore, video game simulators requiring a dedicated GPU are discarded.

Also, environments whose coding language is not very user friendly (such as C/C++) are less preferable from the ones using Python or Lua, as these 2 programming languages are used by the industry nowadays with vast DL resources.

Another feature that was evaluated is realism. Physics and car mobility must be considered as, in the end, the most fidelity to the real world prevails.

Finally, the software costs are taken into balance having in mind that free software is preferable to paid options.

For the evaluation process, the following video game simulators were listed:

- CARLA (CARLA Team, 2017): a free drive simulator that was made to enable the manufacturing process of self-driving car systems. It has few features to enhance realism in its environment (such as the Traffic scenarios simulation), and it requires a minimum of 6 GB GPU. The development language is Python.
- CARLO (Biyik, 2019): inspired in CARLA but designed for machines with less hardware power, this 2D open-source car driving simulator does not require a dedicated GPU and was made in Python. It does not provide much realism, but it is way simpler to use compared to more robust environments.
- GTA V (Rockstar Games, 2013): one of the most successful videogames of all time, this realistic and proprietary software requires 1 GB GPU. To use it as a driving simulator, a modification (mod) written in Python or C++, must be present additionally.
- Roblox (Roblox Corporation, 2006): another case of success in the video game industry, this free-to-play simulation does not require a dedicated GPU. It uses the Lua language for development and has some physics concepts in its environment.
- TORCS (Wymann, 2001): an open-source car simulator software which an enlightened physical model. It does not require a dedicated GPU and uses C/C++ as a scripting language.
- Unity (Unity Technologies, 2005): A partially-free game engine with a very robust physics module. Although it does not require a GPU it is highly recommended to use one. The environment uses the C# language.

Driving Simulator	Free to Use?	Language	Environment	Needs GPU?
CARLA	Yes	Python	Realistic	Yes
CARLO	Yes	Python	Non-realistic	No
GTA V	No	Python	Realistic	Yes
Roblox	Yes	Lua	Realistic	No
TORCS	Yes	C/ C++	Realistic	No
Unity	Yes	C#	Realistic	No

Table 2: Simulators Comparison

Having the features compared among the simulators, Roblox was selected as it is the best fit according to the criteria. Developed by David Baszucki as free software, it can be used as a learning platform (Wardhana, 2021), it is a promising environment for the metaverse (Clark, 2021), and has a realistic physics engine requiring low-budget computer power. Also, it has a front-end development engine (Roblox Studio) that uses Lua, a friendly script language widespread among the industry.

For this research, the simulation is intended to be realistic in terms of vehicle dynamics and physics. For that, CARLO is not suitable once it is the only 2D environment among all others listed above, although it is very friendly and easy to use.

CARLA and the GTA V demand a dedicated GPU, which is not available in the machine used in this research. Both of them require a very robust computer and one is not at disposal.

GTA V is also a paid solution with a non-friendly interface as a mod API must be in place to manipulate the game variables, making it less preferable than the others

Finally, TORCS and Unity use C/C++ and C# in their front-end environments respectively, which makes the development tasks a bit harder compared to other script languages like Lua or Python.

3.2. Deep Learning Technique Review

In the Literature Review section, a few ML techniques were reviewed on regards autonomous driving applications. They are the following:

- CNN: The most common DL method to deal with image processing. This technique uses a filter (convolution kernels) to read the inputs, providing an aggregated interpretation of the same.
- DNN: The traditional algorithm for DL, is widespread through most simulation software.
- DQN: A method mixing RL with DL, where the algorithm uses a NN instead of using a Q-table, mapping the NN input states to pairs like the action and the Q-value.
- H-REIL: A technique to combine both IL (a method that tries to replicate human behaviour in a given task) with RL (a method that rewards desired behaviours and punishes undesired ones).

As the scope of this research is about DL techniques, H-REIL is discarded as it does not relate directly to that. Also, it is way more complex and Roblox does not have any native or external libraries about it.

CNN is recommended to be used with images and it is a popular approach as demonstrated in the Literature Review section. However, most applications employing it require a dedicated GPU which is not available. Therefore, this method is not considered in this project.

DQN is a bit complex for deployment and Roblox does not have any references or native libraries related to DQN. At the beginning of this research, no reference could be found directly to RL techniques in this software. For that reason, the DQN approach is discarded.

Finally, Roblox does not have any native module for DL techniques. But a developer called Kironte (2020) built an entire NN library for Roblox, enabling the use of DNN in its IDE without any external components, making development tasks less complex. Also, the documentation website was made in a user-friendly way, allowing a more autonomous interaction with the tool rather than having to rely on external support. With these considerations, the DNN approach was selected as the DL technique for training the autonomous driving agent in the simulation.

3.3. Variables Review

Having in mind that DNN was selected to be deployed in the Roblox environment, an approach with the application getting sensor data from a vehicle and feeding it into a DNN with backpropagation for learning purposes is preferable. For that, the environment needs to provide data related to the car's sensor like a RADAR (a device that uses radio waves to measure distances between objects). In Roblox, it is possible to calculate the distance of objects with the Raycast function. Therefore, a method similar to Fathi and Izadi (2020) can be observed as this function can get distances from the sides, front, and diagonals (between front and sides).

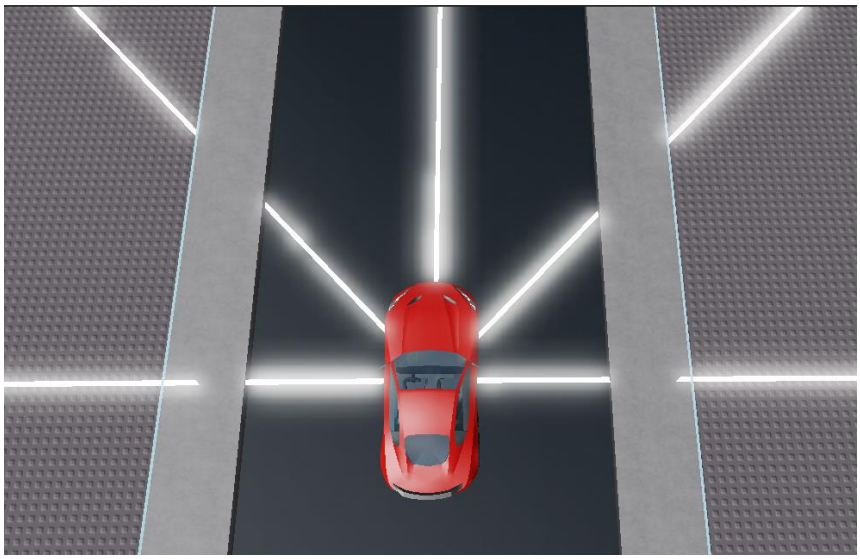

Figure 5: Vectors of the 5 Raycasting on the RADAR simulation from the vehicle displayed in neon white

The 5 distances can be calculated from the car through an object or place. For this project, the Sidewalk was the place to get the distance with. Therefore, the following inputs are considered:

- Distance between the car and the nearest sidewalk tagged by the left ray.
- Distance between the car and the nearest sidewalk tagged by the left-front ray.
- Distance between the car and the nearest sidewalk tagged by the front ray.
- Distance between the car and the nearest sidewalk tagged by the right-front ray.
- Distance between the car and the nearest sidewalk tagged by the right ray.

3.4. Environment Setup

To build the environment, Roblox Studio (Roblox Corporation, 2006) was used as the IDE to develop the script and the virtual places.

The algorithm consists in applying an ego car with a set of pre-stabilised values to train a DNN via backpropagation. Those values are based on the values collected by the Raycasting function which returns the distance between two objects or places. A script was built to make decisions using the normalized values from each distance as parameters. At the end of the process, it returns a value between -1 and 1 to be used to steer the vehicle to one of the sides (or to not steer at all) based on the distances from the sidewalks.

The car population defines how many cars' attempts the simulation run for both train and test sets, and the train/ test split value balance the population in each of them. The total cars used for training is called epochs (Goodfellow et al, 2016). After the epochs' number is reached, the DNN starts to calculate the values based on the inputs from the distances and takes into consideration the learning phase using the backpropagation.

The code also checks if the vehicle hit a sidewalk. In an event of the car collides with the sidewalk, it is destroyed, and another one is created until the count reaches the car population value. Every time a vehicle is destroyed, the script displays a message with the information for the given car such as the car number, how many seconds the car was alive, the distance travelled, and the aggregated cost value of all iterations of the given car.

Each epoch has a limit of max iterations. When this limit is reached, the vehicle is destroyed as well. Each iteration might take 70 milliseconds approximately, and its counter is restarted every time a vehicle is created. Whenever it happens, the system displays a message informing that the max iterations were reached.

The car speed is controlled by the variation of the throttle. Whenever the vehicle is close to a sidewalk in the front, the throttle decrease until each reaches the minimum throttle value or once the car is not close to a sidewalk in its front. Also, the script tries to reach the velocity goal set by decreasing and increasing the throttle.

Applying the Roblox NN Library (Kironte, 2020), a DNN has the following parameters:

- inputNamesArray: An array with the name of all input nodes.
- numberOfLayers: number of hidden layers in the NN.
- numberOfNodesPerLayer: Number of nodes in the hidden layers
- outputNamesArray: An array with the names of all input nodes.
- customSettings: a dictionary with the following parameters:
- Optimizer: the chosen optimizer algorithm for the DNN.
- HiddenActivationName: Activation function for the hidden layers.
- OutputActivationName: Activation function for the output layer.
- LearningRate: The value of the learning rate.

The following constants were used, and their value can be changed for tunning purposes:

- Car population: number of cars to run.
- Max iteration: number of maximum iterations before a car is to be destroyed.
- Train test split: the percentage of the overall population for the test set.
- Velocity goal: the ideal speed in Km/h while not in a curve.
- Max velocity: the maximum speed that can be attained by the vehicle.
- Velocity at curve: the ideal speed at which the car makes the turn.
- Initial throttle: the direction of movement to control the car acceleration.
- Initial torque: how fast the car can reach the maximum speed.

All the code was developed in a single script with some functions to deal with each part of the program. The following functions have been developed by the author of this research:

- NormalizeNum: return a normalized number divided by 100. This function was used to normalize values such as distance between objects and the vehicle.
- StartCar: initiate the car with the initial parameters (torque, steer, speed, among others). It does not return values.
- UpdateCar: update the values of a car. This function is used by the algorithm to update the steering value of a car. Also, it controls the speed of the car by changing the throttle, decreasing the same while it is closed to a front obstacle, and according to the values of minimum and maximum throttle. It does not return values.
- RadarEmulation: return a value for the distance between a car part and the sidewalk. If the function does not find a sidewalk, it returns a null (nil) value.
- GetRadarDistances: return an array with the normalized values for the distance of the 3 inputs: front, front-right and front-left. The format of the outcome of this function corresponds to the input of the DNN input parameters and is also used as input for the training function.

- CheckCar: verify if a car exists. If not, it clones a car for the model in the storage and reset or increment some global variables. It returns the cloned car (or the existing car if it already exists).
- TrainPolicy: check the distances from the inputs and make a decision to turn right, go straight or turn left. It returns a value between -1 and 1, which are to steer the car to the right or left, respectively.

Two circuits were created: track 01 (a rectangular one) and track 02 (an “M” circuit). The circuits have been developed in the Roblox Studio IDE as well, in the graphical interface. Both circuits correspond to a raceway with concrete grey sidewalks on each side and a black carriageway among them.

While track 01 has 330 metres of length, track 02 has 380 metres for its full lap. For both places, the width of the carriageway is 10 metres, which is in the recommended range of 7 and 11 metres by the Transport Infrastructure Ireland for urban roads (Maher, 2015).

Figure 6: Track 01 (Rectangular Circuit)

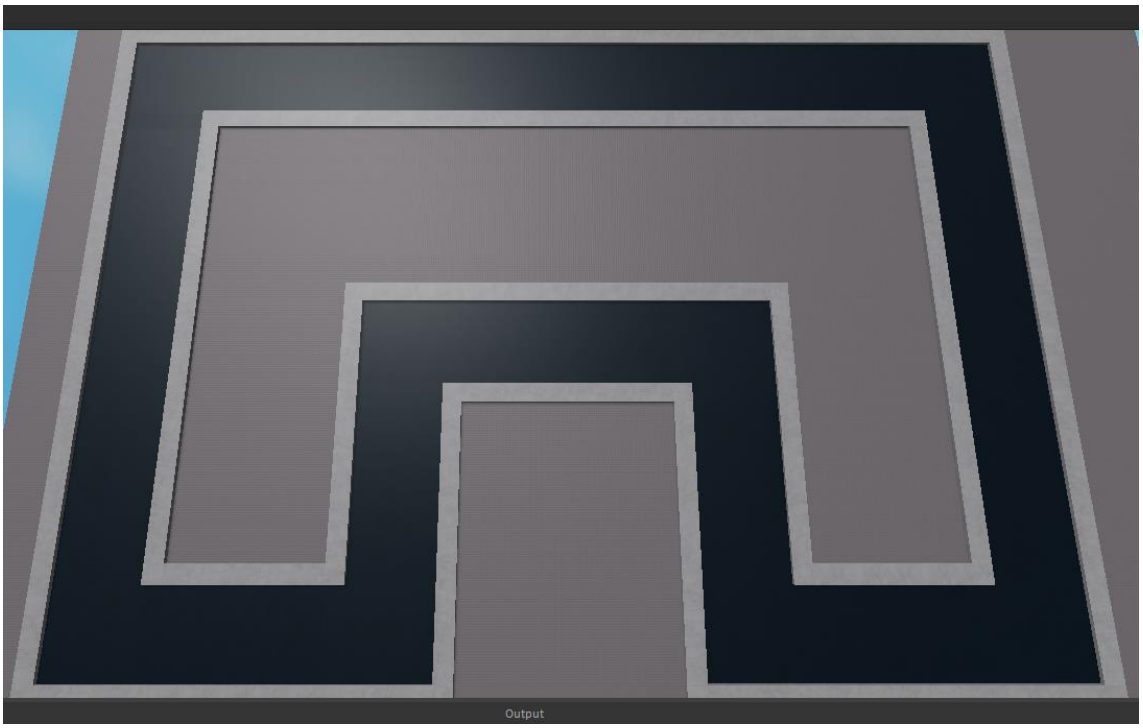

Figure 7: Track 02 ("M" Circuit)

Finally, the free Sports Car model developed by Roblox was used for the vehicles. The model had to be modified for being able to accept commands without a passenger/ player inside of it. While red was the colour applied for cars in the test set, blue was the chosen one for vehicles in the learning phase.

3.5. Methodology Summary

This chapter presented the steps used to guide the project. First, the video game simulators were compared, evaluated and Roblox was selected as the best fit for the project. Then, the DL techniques used in the Literature Review section were analysed and the DNN approach was chosen. After that, the input variables have been verified to be applied in the selected DL method. Finally, it was presented how the environment is set up along with its features.

4. Implementation

In this chapter, the analysis of the parameters used in the project and the results of the learning attempts are demonstrated. Relevant results are reported concisely and objectively in a logical order with the support of tables and pictures to illustrate specific findings. As the nature of this work is empirical, results are informed before the discussion of their meanings. For that, it is important to notice that interpretations are not included in this section once they are demonstrated in the next chapter.

4.1. Parameter's Setup & Tunning

The research was performed setting parameters for the following constants:

- Max iteration: 10,000
- Velocity goal: 25
- Max velocity: 60
- Velocity at curve: 20
- Initial throttle: 0.25
- Initial torque: 200,000

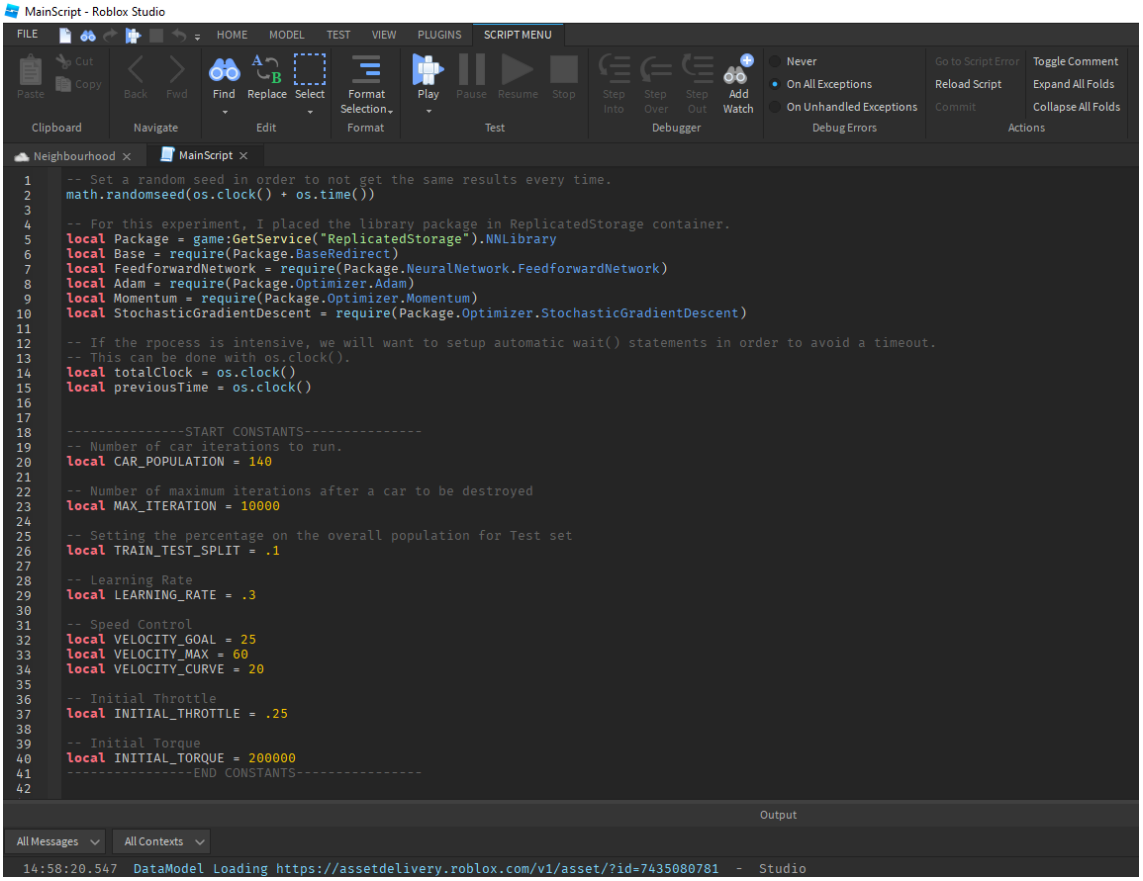

Figure 8: Screen Capture of Roblox Studio IDE with some Parameters and their Values

Initially, attempts with lower values for max iteration (e.g.: 5000, 3000) have been tried but the same have been discarded since the results were very poor.

The DNN was set up as the following:

- Input nodes: 3 (Left-Front distance, Front distance, and Right-Front distance).
- Hidden layers: 2 (with different nodes in each experiment).
- Output node: 1 (steering direction with 1 being 45° to the left, 0 being up-front, and -1 being 45° to the right).
- Optimizer: Momentum (a faster variation of the Stochastic Gradient Descent algorithm) (Goodfellow et al, 2016).
- Activation function for the hidden layers: LeakyReLU (a refined variety of the ReLU algorithm which gives a resolution to the dying problem with a small positive slope for values lesser than zero) (Goodfellow et al, 2016).
- Activation function for the output layer: Tanh (also known as Hyperbolic Tangent, this activation function assumes values from -1 to 1, which matches the range of expected values of the output) (Haykin, 2009).
- Learning rate: 0.3.

Neighbourhood × MainScript ×

```lua

-- This setting dictionary contains whatever customizations you want on your neural network.
-- Each setting has it's own default and is completely optional.
local setting = {
    Optimizer = Momentum.new();

    -- We want to accept negative inputs so a LeakyReLU will do.
    -- Also, LeakyReLU works better with backpropagation compared to ReLU
    HiddenActivationName = "LeakyReLU";

    -- The output is between -1 and 1 for Tanh.
    -- This is exactly what we want for Steer Direction (1 for left, -1 for right).
    OutputActivationName = "Tanh";

    LearningRate = LEARNING_RATE;
}

-- Create a new network with 3 inputs, 2 layers with 7 nodes each and 1 output "steerDirection"
local net = FeedforwardNetwork.new({"front", "frontRight", "frontLeft"}, 2, 7, {"steerDirection"}, setting)

local backProp = net:GetBackPropagator()
```

Figure 9: Screen Capture of Roblox Studio IDE with the DNN Parameters

In initial attempts, the DNN was set up with 5 input nodes, but the 2 nodes from the sides (right and left) have been removed as they have not been used in the training algorithm. Hence, the input parameters used for the experiment are left-front, front and right-front.

Approaches with other learning rates (e.g.: 0.7, 0.6, 0.1 etc.) were tested as well but the outcome was far from the ideal and, for that reason, they have been replaced for the 0.3 value, which was the value with the better results among several attempts.

4.2. Results

For the work of this research, the results are based on how many cars were able to complete a lap in the circuit. For track 01, any attempt with more than 330 metres can be considered successful while it is needed 380 metres for a triumphant outcome at track 02.

Two (2) scenarios were set up for that: the first one (scenario 1) with 140 cars of the total population and a split for the train-test sets of 10%, and the second one (scenario 2) with 50 cars of the total population and a split for the train-test sets of 20%. The idea behind the scenarios was to have a very robust but computer resources demanding training approach (scenario 1) which can make the DNN have plenty of information to learn how to drive, and another one (scenario 2) with less population and a higher percentage for the test set, which would demand fewer computer resources.

4.2.1. Scenario 1

The first scenario of tests was made with eight (8) different attempts initially, each of them with different hidden layer layouts. As each of the attempts had 140 cars in total and the split for the train-test sets was 10%, it left the training set with 126 cars (epochs) and the test set with 14 cars.

Track 01

The attempts in track 01 took an average time of 7 hours and 58 minutes to complete all the cars (training and test sets), and the test set results can be seen in the following tables:

Layout	5 nodes		6 nodes		7 nodes		8 nodes	
Attempt	**seconds**	**metres**	**seconds**	**metres**	**seconds**	**metres**	**seconds**	**metres**
1	33.92	225.58	25.38	147.61	425,82	3.314,46	49,74	321,10
2	63.14	481.06	36.44	303.25	79,82	601,38	121,97	847,06
3	34.38	253.89	24.95	97.92	430,19	3.460,72	26,81	163,14
4	34.52	238.07	27.76	140.06	475,51	3.653,72	26,41	165,71
5	44.01	313.32	17.87	116.16	29,39	201,82	20,81	105,25
6	44.96	334.18	39.79	261.44	70,34	520,49	34,00	214,16
7	25.19	159.97	24.87	137.48	157,62	1.229,14	36,24	229,49
8	33.67	240.85	33.57	261.68	527,12	4.014,89	119,33	855,06
9	37.52	264.01	30.46	133.53	155,46	1.192,24	26,06	163,53
10	36.39	262.95	25.43	137.56	65,29	521,65	18,59	95,94
11	37.54	266.53	20.18	136.54	110,85	854,95	27,21	165,27
12	35.72	244.79	26.37	174.42	108,99	854,67	66,88	438,13
13	37.95	266.84	24.06	95.54	26,40	171,23	29,82	164,56
14	113.68	839.23	18.92	112.83	25,82	169,77	19,71	97,34
Average	**43.76**	**313.66**	**26.86**	**161.14**	**192,04**	**1.482,94**	**44,54**	**287,55**
Ended Laps	**3**		**0**		**11**		**3**	

Table 3: Results for the Test Set on 10% of Total Population (140 Cars) in Track 01 – Part A

Layout	9 nodes		10 nodes		12 nodes		24 nodes	
Attempt	**seconds**	**metres**	**seconds**	**metres**	**seconds**	**metres**	**seconds**	**metres**
1	24.85	139.05	18.59	94.70	22.73	139.64	12.44	52.22
2	63.75	457.26	18.88	95.11	31.86	204.45	14.69	68.04
3	40.62	314.60	25.55	141.75	142.90	1.099.87	12.27	53.33
4	53.46	385.14	17.67	94.43	17.07	93.33	12.27	50.53
5	49.70	378.55	17.98	94.51	16.85	94.48	12.70	54.11
6	26.29	146.79	15.85	76.88	16.99	93.56	12.74	54.04
7	21.98	138.21	17.58	94.55	51.29	333.00	12.20	58.66
8	31.06	225.01	15.10	75.66	27.84	161.28	12.14	49.85
9	25.62	138.55	37.88	223.24	24.44	138.11	12.98	54.21
10	53.99	428.96	18.52	94.94	22.81	142.32	13.40	53.04
11	34.32	214.08	18.87	95.64	28.24	181.97	13.59	52.69
12	37.74	308.72	40.07	246.33	75.84	554.97	17.93	52.67
13	33.58	193.24	18.49	95.41	59.00	429.82	17.47	53.38
14	22.19	141.58	18.37	94.87	23.53	146.69	14.54	53.81
Average	**37.08**	**257.84**	**21.39**	**115.57**	**40.10**	**272.39**	**13.67**	**54.33**
Ended Laps	**4**		**0**		**4**		**0**	

Table 4: Results for the Test Set on 10% of Total Population (140 Cars) in Track 01 – Part B

It is possible to notice that the attempts with 6, 10 and 24 nodes in the hidden layer performed worse compared to other ones. The best attempt was the one with 7 nodes which had 11 of 14 cars completing the circuit.

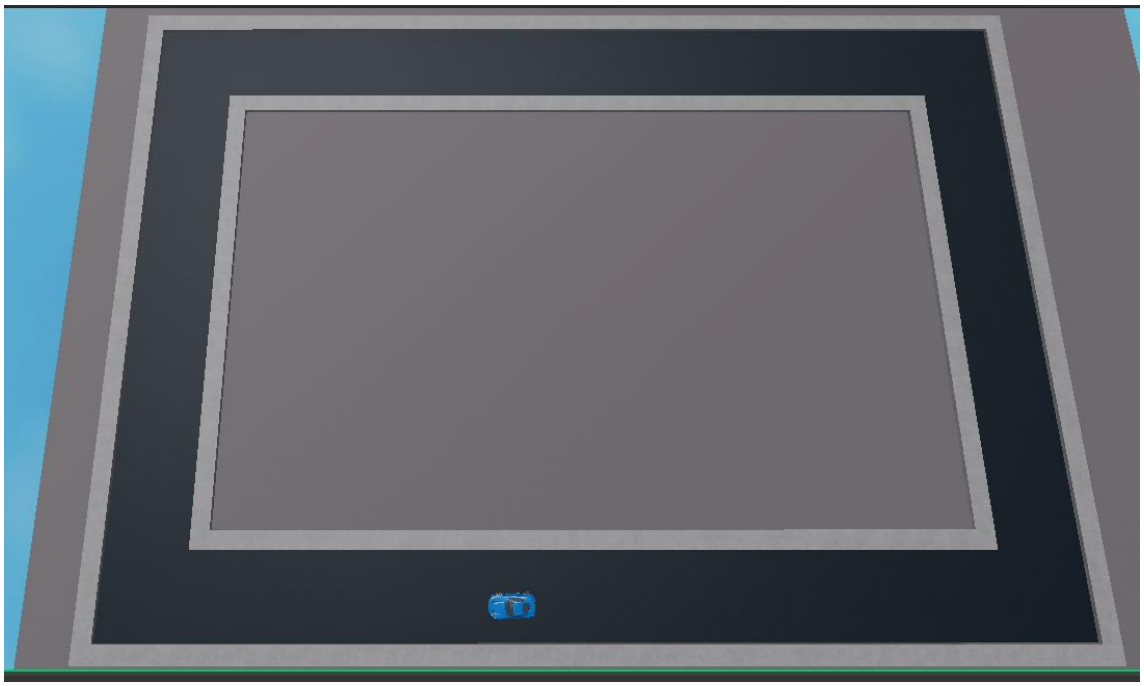

Figure 10: Training Car in Track 01

Track 02

For track 02, a decision was made to only replicate hidden layer layouts with at least 1 successful outcome in track 01. With that, attempts with no cars completing laps in track 01 (6, 10 and 24 nodes) have not been tested. This decision was due to the complexity of the track 02 circuit for the DNN to handle, which is higher than the one of track 01.

The test set results took a total time of 6 hours and 7 minutes to complete all the cars and can be seen in the below tables:

Layout	5 nodes		7 nodes		8 nodes	
Attempt	**seconds**	**metres**	**seconds**	**metres**	**seconds**	**metres**
1	35.51	229.77	25.61	97.21	12.88	70.31
2	37.37	230.38	39.69	230.82	11.31	59.98
3	29.18	175.88	31.27	230.77	12.50	70.53
4	35.91	229.09	27.84	163.37	15.32	70.60
5	35.41	229.63	31.89	149.93	80.39	454.20
6	35.79	222.27	22.46	145.48	13.59	68.73
7	36.55	222.24	32.19	238.26	27.64	146.71
8	35.84	225.29	39.07	234.72	12.53	50.00
9	36.41	229.78	34.20	167.47	12.64	71.71
10	28.65	175.95	33.04	226.22	12.02	67.39
11	35.57	222.78	32.36	230.89	11.27	54.19
12	37.76	229.93	109.79	653.31	27.56	165.08
13	36.75	222.09	30.81	165.54	13.76	50.72
14	26.80	163.31	34.84	231.10	11.25	49.20
Average	**34.54**	**214.89**	**37.50**	**226.08**	**19.62**	**103.53**
Ended Laps	**0**		**1**		**1**	

Table 5: Results for the Test Set on 10% of Total Population (140 Cars) in Track 02 – Part A

Layout	9 nodes		12 nodes	
Attempt	seconds	metres	seconds	metres
1	25.27	67.57	10.31	13.58
2	60.22	366.93	9.07	11.97
3	18.96	74.34	9.79	12.42
4	21.19	94.37	9.38	20.96
5	19.27	94.88	7.67	21.93
6	13.55	73.41	7.80	22.05
7	12.44	72.53	7.72	22.00
8	14.90	96.52	7.01	20.71
9	13.79	76.69	7.99	27.31
10	18.89	72.72	9.03	24.73
11	65.77	366.89	9.74	13.81
12	12.57	73.45	11.70	24.26
13	18.92	96.49	9.21	11.61
14	20.09	95.93	8.59	13.01
Average	**23.99**	**123.05**	**8.93**	**18.60**
Ended Laps	**0**		**0**	

Table 6: Results for the Test Set on 10% of Total Population (140 Cars) in Track 02 – Part B

Despite the results are not good as the ones observed for track 01, it is possible to notice that some cars completed the lap in the attempts with 7 and 8 nodes.

4.2.2. Scenario 2

The second scenario of tests was made with the same eight (8) attempts as well, each of them with 50 cars, and the split for the train-test sets was 20% (instead of 10% only in the previous scenario). With that, the training set had a population of 40 cars while the test set had 10 cars.

Track 01

In this scenario, the attempts took an average time of 1 hour and 27 minutes to complete all the cars in track 01, and the test set results can be seen in the below tables:

Layout	5 nodes		6 nodes		7 nodes		8 nodes	
Attempt	seconds	metres	seconds	metres	seconds	metres	seconds	metres
1	75.17	446.93	121.31	600.32	358.76	2.412.86	20.25	68.65
2	28.12	115.45	34.14	93.55	210.60	1.585.28	19.40	70.93
3	12.85	19.31	80.10	600.40	147.05	1.167.33	13.90	74.93
4	23.25	113.67	35.58	183.88	107.76	831.66	12.01	69.13
5	64.31	397.69	75.82	498.73	39.13	202.17	219.30	1.736.73
6	21.38	71.04	106.44	854.97	114.01	757.55	12.07	67.70
7	14.84	51.95	32.36	163.64	82.96	519.85	14.61	63.35
8	11.36	52.9	122.61	935.20	33.28	202.10	20.84	75.73
9	8.83	35.32	183.20	1.517.25	175.38	1.253.89	19.61	69.13
10	36.29	221.08	24.62	171.17	100.16	427.16	20.23	70.19
Average	**29.64**	**152.53**	**81.62**	**561.91**	**136.91**	**935.99**	**37.22**	**236.65**
Ended Laps	**2**		**6**		**8**		**1**	

Table 7: Results for the Test Set on 20% of Total Population (50 Cars) in Track 01 – Part A

Layout	9 nodes		10 nodes		12 nodes		24 nodes	
Attempt	seconds	metres	seconds	metres	seconds	metres	seconds	metres
1	27.40	190.72	26.19	109.76	6.97	8.53	16.27	67.81
2	28.61	160.68	25.54	110.65	5.04	11.47	19.91	67.34
3	41.00	247.26	35.70	297.94	5.09	12.24	18.05	67.14
4	20.74	138.83	35.80	116.91	4.94	11.39	12.18	68.47
5	92.16	718.37	29.12	127.51	4.92	11.30	12.05	67.08
6	32.98	163.54	36.13	300.07	4.91	10.28	12.41	67.79
7	21.63	106.41	28.75	126.25	5.15	12.57	18.85	68.27
8	70.94	506.88	40.17	132.92	5.39	12.83	18.15	66.56
9	31.11	207.28	27.04	188.49	5.39	12.45	14.24	66.58
10	24.07	138.74	33.76	215.21	5.05	11.64	11.93	66.02
Average	**39.06**	**257.87**	**31.82**	**172.57**	**5.29**	**11.47**	**15.40**	**67.31**
Ended Laps	**2**		**0**		**0**		**0**	

Table 8: Results for the Test Set on 20% of Total Population (50 Cars) in Track 01 – Part B

It is possible to notice that the attempts with 10, 12 and 24 nodes in the hidden layer performed worse compared to the attempts with fewer nodes. Similar to scenario 1, the attempt with 7 nodes was the best one, when 8 of its 10 cars finished the lap.

Track 02

The same decision applied previously to exclude attempts with no cars completing laps in track 01 was applied for track 02 in this scenario. With that, attempts with 10, 12 and 24 nodes have not been replicated in track 02.

The test set results took an average time of 1 hour and 41 minutes, and can be found in the following tables:

Layout	5 nodes		6 nodes		7 nodes	
Attempt	seconds	metres	seconds	metres	seconds	metres
1	14.28	32.77	159.94	545.14	43.92	227.47
2	13.06	32.51	56.08	165.27	42.35	226.14
3	8.82	35.12	47.82	203.79	50.66	219.43
4	12.83	70.25	56.74	162.44	35.59	222.40
5	8.52	36.84	24.78	161.30	55.33	218.81
6	8.52	36.21	84.52	231.35	57.75	226.80
7	11.03	33.01	51.78	226.78	67.77	229.45
8	11.83	32.37	52.03	163.34	38.36	200.79
9	14.42	32.62	44.95	206.86	50.64	165.18
10	13.23	32.74	53.37	164.05	38.11	224.19
Average	**11.65**	**37.44**	**63.20**	**223.03**	**48.05**	**216.07**
Ended Laps	**0**		**1**		**0**	

Table 9: Results for the Test Set on 20% of Total Population (50 Cars) in Track 02 – Part A

Layout	8 nodes		9 nodes	
Attempt	seconds	metres	seconds	metres
1	19.20	125.06	40.38	219.24
2	33.93	232.07	29.20	218.15
3	24.04	95.66	35.68	176.95
4	36.46	229.89	23.08	121.48
5	31.34	226.05	33.01	217.58
6	32.23	177.89	34.88	222.05
7	33.15	227.33	40.32	217.82
8	20.44	139.65	33.15	218.52
9	43.88	228.27	30.89	217.22
10	33.53	228.21	24.92	133.72
Average	**30.82**	**191.01**	**32.55**	**196.27**
Ended Laps	**0**		**0**	

Table 10: Results for the Test Set on 20% of Total Population (50 Cars) in Track 02 – Part B

It is feasible to see that the attempt with 6 nodes had a successful car, although most of the results in track 02 are far behind the ones in track 01.

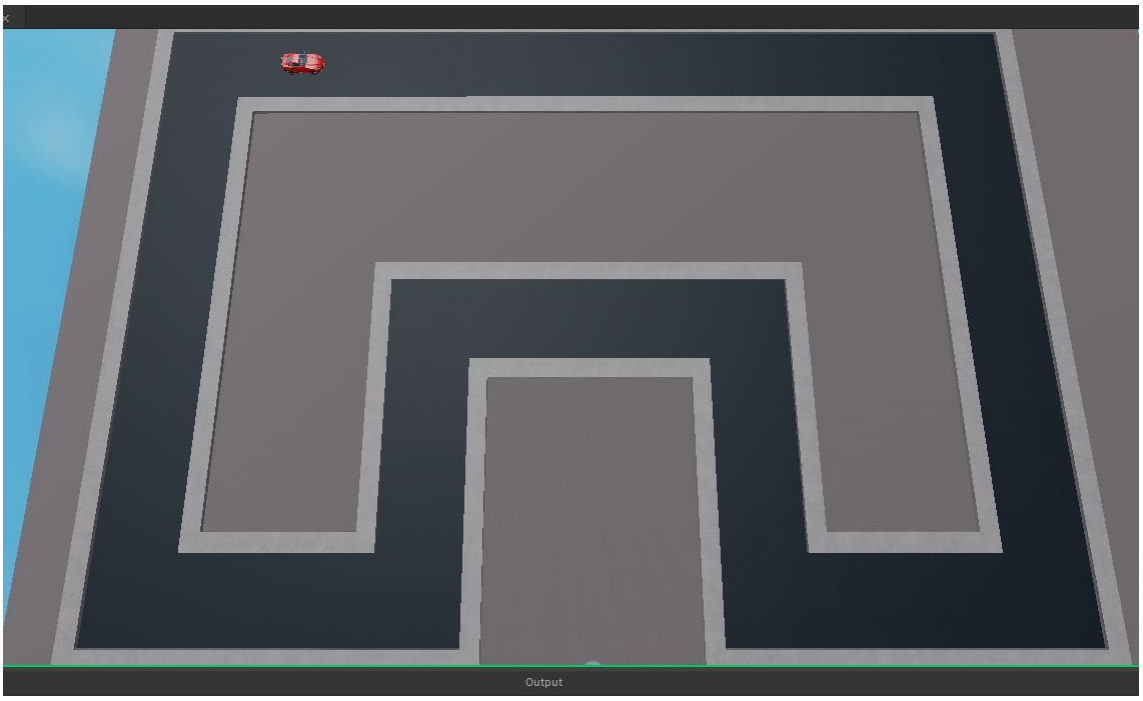

Figure 11: Testing Car in Track 02

4.3. Implementation Summary

This chapter presented the parameter tuning exploration and the discussion of the experiment's results. First, the description and values of constants and variables used in the experiment were informed, explaining how the environment and the DNN were set up. Then, the test scenarios were described having into account the length of each circuit, and their results were exhibited showing the number of cars that completed laps for each attempt with distances and the time for the same.

Although some insights were informed, the detailed interpretations of the results and their significance on the purpose of this research will be addressed in the next chapter.

5. Discussion

In this section, an analysis related to the meaning, importance and relevance of the results are presented, showing the significance of the same. Correlations, patterns, and relationships among the data are identified and described, and a discussion of whether the results support the hypotheses is presented as well. This chapter also tries to answer how the application of DL in games improve autonomous driving, providing arguments to the conclusion. Finally, recommendations for practical implementation or further research are given in the end.

5.1. Evaluation

The results for Scenario 1 show that most of the attempts had cars that were able to finish the lap successfully on track 01, with the result of 11 in 14 cars (79%) completing the circuit in the attempt with 7 being the best among the others. On the other hand, the performance of the vehicle in track 02 was not as good as the previous one, which only 1 car (7%) completing the circuit for attempts with 7 and 8 nodes.

Similarly, the results for Scenario 2 demonstrate that some cars completed the lap in track 01 for the majority of the attempts. The performance of the attempts with 7 nodes was slightly better than the one in the previous scenario, with 8 in 10 cars (80%) finishing the lap. But the results for track 02 were very poor compared with track 01 as only 1 car (10%) in the attempt with 6 nodes completed the circuit.

Overall, the results show that the DNN is learning from the training set, although the accuracy is not very high. It is possible to notice that the number of iterations can interfere with the results as it seems that the scenario with less iteration had a better performance compared with the other one for a simple circuit (track 01). The results are dubious for a more complex circuit (track 02) as the performance is poor for both scenarios.

The results for track 01 in both scenarios demonstrated that the technique had a good performance even compared to the ones from projects using a similar technique, such as Albelihi (2014) and Cao et al (2020). However, if a more complex circuit is taken into consideration (like track 02), the outcome is behind the expected.

It is important to mention that approaches with different parameter values (Learning Rate, iterations, etc.) have been tried before, but the results were not satisfactory as the ones presented in the above sections.

5.2. Scope and Limitations

Although some experiments addressed scenarios using more advanced ML techniques, such as DQN with CNN (El Sallab et al, 2017) and Multiagent DRL (Wong et al, 2021), the scope of this work is related to more traditional DL techniques like DNN.

A computer more than 10 years old (Intel® Core™ i5-480M, with 6 GB RAM, running Windows 10) is far from the ideal to process algorithms requiring heavy tasks. The performance of the computer can interfere in the video game environment causing lag (a delay between the action of the DNN inputs and the response of the device running the simulator) and other random events which might compromise the algorithm's result.

Although Roblox has almost 20 dedicated people in its support team, some bugs are constantly reported every day by users. They release updates on a timely basis, but some fixes might not achieve all scenarios, in special the ones not related to gameplay. Also, the NN library developed by Kironte (2020) is in an experimental phase when the tests of this research took place, and it might have improvements in the future. Finally, the server mode (that means using Roblox Servers' infrastructure to run the simulation) has a timeout limit of 20 minutes, which means that users have to "move" their character in the simulation at least once before this time exceeds.

The attempts were very time consuming and errors in the device lead to the loss of several days of work. Some attempts in Scenario 1 (total population of 140 cars) took more than 11 hours to finish and the processing task was done overnight when no person was using the device. Tries with larger populations had to be discarded as it would take much more time and the risk of crashing could lead to missing the project deadlines.

5.3. Conclusions

The focus of this research is to verify how the application of DL in games would improve autonomous driving. The simulations produced were not broad enough to show the real benefits or disadvantages of the application of video games in autonomous driving. Attempts with more cars (larger populations) running with different parameters could be done divergently. However, the developed algorithm demonstrated that such environments can be used to apply ML techniques and to train AI algorithms for self-driving vehicles.

The DNN technique is widespread among most of the platforms, which can be seen even in software not related to data analytics (such as Roblox). However, it is important to notice that not using image detection techniques with ML (like CNN) prevent the algorithm to be the central component of an autonomous driving AI. On the other hand, this technique is much faster to be trained compared to more robust techniques such as DRL and CNN and requires fewer computing resources, which makes it a good option to complement existing AI algorithms.

5.4. Future Work

As mentioned previously, this technique can be used as a complementary module for supporting autonomous driving (such as parking assist, object-collision detector, among others).

Further research using CNN (with image processing) or DRL (enabling a more autonomous learning AI) techniques is needed to establish a more accurate and more autonomous AI algorithm inside Roblox. This platform has demonstrated its capabilities as a friendly IDE for developers and can be used for this research. Modelling a more complex driving scenario will imply having more computer resources available, and Roblox servers are a good fit for that (except for the timeout limitation, as stated before). And with the new metaverse resources available in this environment, new features could be developed using ML techniques.

Future studies should take into account the hardware (in particular if the focus is about image processing), timing (timely base of tests), and software limitations (knowing bugs, timeout, etc.). With that, a more complex scenario can be built, adding features like lane control, other car/ drivers, pedestrians, traffic lights, etc., which can be addressed with more computer power.

Finally, this research has been developed in a platform where young people use for leisure purposes daily. The techniques adopted here can mix the joy of the ones with learning opportunities and could inspire a new generation of data analytics developers.

6. References

ALBELIHI, K. (2014). THE GAZELLE ADAPTIVE RACING CAR PILOT. *[MASTER OF SCIENCE THESIS, INDIANA UNIVERSITY SOUTH BEND, SOUTH BEND, INDIANA, USA].* AVAILABLE ONLINE AT: https://clas.iusb.edu/math-compsci/_prior-thesis/KAlbelihi_Thesis.pdf (ACCESSED: 20 OCTOBER 2021)

ALMASI, P. MONI, R. GYIRES TOTH, B. (2020). ROBUST REINFORCEMENT LEARNING-BASED AUTONOMOUS DRIVING AGENT FOR SIMULATION AND REAL WORLD. *IN 2020 INTERNATIONAL JOINT CONFERENCE ON NEURAL NETWORKS (IJCNN)*, PP 1-8 AVAILABLE ONLINE AT: https://arxiv.org/pdf/2009.11212.pdf (ACCESSED: 22 JUNE 2021)

BABIKER, M. ELAWAD, M. AHMED, A. (2019). CONVOLUTIONAL NEURAL NETWORK FOR A SELF-DRIVING CAR IN A VIRTUAL ENVIRONMENT. *2019 INTERNATIONAL CONFERENCE ON COMPUTER, CONTROL, ELECTRICAL, AND ELECTRONICS ENGINEERING (ICCCEEE19)*. AVAILABLE ONLINE AT: https://ieeexplore.ieee.org/document/9070826 (ACCESSED: 08 OCTOBER 2021)

BENGIO, Y. (2012). PRACTICAL RECOMMENDATIONS FOR GRADIENT-BASED TRAINING OF DEEP ARCHITECTURES. AVAILABLE ONLINE AT: https://arxiv.org/abs/1206.5533 (ACCESSED: 15 OCTOBER 2021)

BENGIO, Y. LECUN, Y. HINTON, G. (2021). DEEP LEARNING FOR AI. *COMMUNICATIONS OF THE ACM*, PP 58-65, 64(7). AVAILABLE ONLINE AT: https://dl.acm.org/doi/10.1145/3448250 (ACCESSED: 11 OCTOBER 2021)

BIYIK, E. (2019). CARLO. PROCEEDINGS OF ROBOTICS: SCIENCE AND SYSTEMS (RSS). AVAILABLE ONLINE AT: https://github.com/Stanford-ILIAD/CARLO (ACCESSED: 24 SEPTEMBER 2021)

BRATOSIN, I. PĂVĂLOIU, I. GOGA, N. DRĂGOI, G. (2019). AUTONOMOUS DRIVING IN GAMES. *PROCEEDINGS OF ELSE 2019 THE 15TH INTERNATIONAL SCIENTIFIC CONFERENCE ELEARNING AND SOFTWARE FOR EDUCATION*, PP. 46 52. AVAILABLE ONLINE AT: https://www.proquest.com/docview/2213792036 (ACCESSED: 10 JUNE 2021)

CAO, Z. BIYIK, E. WANG, W. Z. RAVENTOS, A. GAIDON, A. ROSMAN, G. SADIGH, D. (2020). REINFORCEMENT LEARNING BASED CONTROL OF IMITATIVE POLICIES FOR NEAR-ACCIDENT DRIVING. *ROBOTICS: SCIENCE AND SYSTEMS (RSS) 2020*. AVAILABLE ONLINE AT: https://arxiv.org/abs/2007.00178 (ACCESSED: 24 SEPTEMBER 2021)

CARLA TEAM (2017). CARLA - OPEN-SOURCE SIMULATOR FOR AUTONOMOUS DRIVING RESEARCH. *CARLA TEAM*. AVAILABLE ONLINE AT: https://carla.org/ (ACCESSED: 04 SEPTEMBER 2021)

CHAUDHARI, R. DUBEY, S. KATHALE J. RAO, R. (2018). AUTONOMOUS DRIVING CAR USING CONVOLUTIONAL NEURAL NETWORKS. *2018 SECOND INTERNATIONAL CONFERENCE ON INVENTIVE COMMUNICATION AND COMPUTATIONAL TECHNOLOGIES (ICICCT)*, PP. 936-940. AVAILABLE ONLINE AT: https://ieeexplore.ieee.org/document/8473043 (ACCESSED: 17 OCTOBER 2021)

CLARK, P. A. (2021). THE METAVERSE HAS ALREADY ARRIVED. HERE'S WHAT THAT ACTUALLY MEANS. *TIME MAGAZINE*. NOVEMBER 15TH, 2021. AVAILABLE ONLINE AT: https://time.com/6116826/what-is-the-metaverse/ (ACCESSED: 07 JANUARY 2022)

DOSOVITSKIY, A. ROS, G. CODEVILLA, F. LOPEZ, A. KOLTUN, V. (2017). CARLA: AN OPEN URBAN DRIVING SIMULATOR. *PROCEEDINGS OF THE 1ST ANNUAL CONFERENCE ON ROBOT LEARNING*, PP 1-16. AVAILABLE ONLINE AT:

http://proceedings.mlr.press/v78/dosovitskiy17a/dosovitskiy17a.pdf (Accessed: 15 November 2021)

El Sallab, A. Abdou, M. Perot, E. Yogamani, S. (2017). Deep Reinforcement Learning Framework for Autonomous Driving. *IS&T Electronic Imaging, Autonomous Vehicles and Machines 2017*, AVM-023, pg. 70-76. Available online at: https://arxiv.org/abs/1704.02532 (Accessed: 04 July 2021)

Encyclopaedia Britannica (2017). Grand Theft Auto (Video Game). *Encyclopaedia Britannica*. Available online at: https://www.britannica.com/topic/Grand-Theft-Auto-video-game (Accessed: 06 November 2021)

Fathi, K. Izadi, I. (2020). Analysis of Feed Forward Neural Network Autonomous Driving Agents in Games. 2020 6th Iranian Conference on Signal Processing and Intelligent Systems (ICSPIS), pp. 1-6. Available online at: https://ieeexplore.ieee.org/document/9349532 (Accessed: 17 October 2021)

Goodfellow, I. Bengio, Y. Courville, A. (2016). Deep Learning. MIT Press. Available online at: https://www.deeplearningbook.org/ (Accessed: 11 October 2021)

Grigorescu, S. Trasnea, B. Cocias, T. Macesanu, G. (2020). A Survey of Deep Learning Techniques for Autonomous Driving. *Journal of Field Robotics*, vol. 37, no. 3, pp. 362–386. Available online at: https://arxiv.org/abs/1910.07738 (Accessed: 04 July 2021)

Haykin, S. (2009). Neural Networks and Learning Machines. *Prentice-Hall/ Pearson*. Available online at: https://cours.etsmtl.ca/sys843/REFS/Books/ebook_Haykin09.pdf (Accessed: 11 October 2021)

Hennig, C. Kutlukaya, M. (2007). Some Thoughts about the Design of Loss Functions. *REVSTAT-Statistical Journal*, V 5. N 1. pp 19-39. Available online at: https://www.ine.pt/revstat/pdf/rs070102.pdf (Accessed: 15 October 2021)

Idrissi, M. A. J. Ramchoun, H. Ghanou, Y. Ettaouil, M. (2016). Genetic Algorithm for Neural Network Architecture Optimization. *Proceedings of the 3rd IEEE International Conference on Logistics Operations Management*, GOL 2016. Available online at: https://ieeexplore.ieee.org/abstract/document/7731699 (Accessed: 14 October 2021)

Kim, J. Lim, G. Kim, Y. Kim B. Bae, C. (2019) Deep Learning Algorithm using Virtual Environment Data for Self-driving Car. *2019 International Conference on Artificial Intelligence in Information and Communication (ICAIIC)*, pp. 444-448 Available online at: https://ieeexplore.ieee.org/document/8669037 (Accessed: 17 October 2021)

Kironte (2020). Roblox Neural Network Library. *Kironte*. Available online at: https://github.com/Kironte/Roblox-Neural-Network-Library (Accessed: 07 September 2021)

Maher, P. (2015). Cross Sections and Headroom (including Amendment No. 2, dated June 2015). *Transport Infrastructure Ireland - DN-GEO-03036. June 2015.* Available online at: https://www.tiipublications.ie/library/DN-GEO-03036-05.pdf (Accessed: 28 November 2021)

Martinez II, M.A. Sitawarin, C. Finch, K. Meincke, L. Yablonski, A. Kornhauser, A. (2017). Beyond Grand Theft Auto V for Training, Testing and Enhancing Deep Learning in Self Driving

CARS. *TRB 2018 ANNUAL MEETING*. AVAILABLE ONLINE AT: https://arxiv.org/pdf/1712.01397.pdf (ACCESSED: 17 OCTOBER 2021)

MAURER, M. GERDES, J. C. LENZ, B. WINNER, H. (2016). AUTONOMOUS DRIVING: TECHNICAL LEGAL AND SOCIAL ASPECTS. AVAILABLE ONLINE AT: https://link.springer.com/content/pdf/10.1007%2F978-3-662-48847-8.pdf (ACCESSED: 27 JUNE 2021)

MEMON, Q. AHMED, M. ALI, S. (2016). SELF-DRIVING AND DRIVER RELAXING VEHICLE. *2ND INTERNATIONAL CONFERENCE ON ROBOTICS AND ARTIFICIAL INTELLIGENCE (ICRAI)*. AVAILABLE ONLINE AT: https://ieeexplore.ieee.org/abstract/document/7791248 (ACCESSED: 09 AUGUST 2021)

PARR, T. HOWARD, J. (2018). THE MATRIX CALCULUS YOU NEED FOR DEEP LEARNING. AVAILABLE ONLINE AT: https://arxiv.org/abs/1802.01528 (ACCESSED: 14 OCTOBER 2021)

PĂVĂLOIU, I. VASILĂŢEANU, A. GOGA, N. YASSINE, M. M. (2018). LEARNING TO DRIVE CARS IN LARGE CITIES. *THE 14TH INTERNATIONAL SCIENTIFIC CONFERENCE ELEARNING AND SOFTWARE FOR EDUCATION*, VOL 1, PP. 313-319. BUCHAREST. AVAILABLE ONLINE AT: https://www.proquest.com/docview/2038227352 (ACCESSED: 09 AUGUST 2021)

ROBLOX CORPORATION (2006). ROBLOX STUDIO. *ROBLOX CORPORATION*. AVAILABLE ONLINE AT: https://www.roblox.com/create (ACCESSED: 28 AUGUST 2021)

ROCKSTAR GAMES (2013). GRAND THEFT AUTO V. *ROCKSTAR GAMES*. AVAILABLE ONLINE AT: https://www.rockstargames.com/V/ (ACCESSED: 15 NOVEMBER 2021)

RONG, G. SHIN, B. H. TABATABAEE, H. LU, Q. LEMKE, S. MOZEIKO, M. BOISE, E. UHM, G. GEROW, M. MEHTA, S. AGAFONOV, E. KIM, T. H. STERNER, E. USHIRODA, K. REYES, M. ZELENKOVSKY, D. KIM, S. (2020). LGSVL SIMULATOR: A HIGH FIDELITY SIMULATOR FOR AUTONOMOUS DRIVING. *2020 IEEE 23RD INTERNATIONAL CONFERENCE ON INTELLIGENT TRANSPORTATION SYSTEMS (ITSC)*, PP. 1-6. AVAILABLE ONLINE AT: https://ieeexplore.ieee.org/abstract/document/9294422 (ACCESSED: 11 AUGUST 2021)

SCHMIDHUBER, J. (2014). DEEP LEARNING IN NEURAL NETWORKS: AN OVERVIEW. *TECHNICAL REPORT IDSIA,* PP 03-14. AVAILABLE ONLINE AT: https://arxiv.org/abs/1404.7828 (ACCESSED: 11 OCTOBER 2021)

TALPAERT, V. SOBH, I. KIRAN, B. R. MANNION, P. YOGAMANI, S. EL-SALLAB, A. PEREZ, P. (2019). EXPLORING APPLICATIONS OF DEEP REINFORCEMENT LEARNING FOR REAL-WORLD AUTONOMOUS DRIVING SYSTEMS. *14TH INTERNATIONAL JOINT CONFERENCE ON COMPUTER VISION, IMAGING AND COMPUTER GRAPHICS THEORY AND APPLICATIONS*. AVAILABLE ONLINE AT: https://arxiv.org/abs/1901.01536 (ACCESSED: 10 JUNE 2021)

TOBIN, J. FONG, R. RAY, A. SCHNEIDER, J. ZAREMBA, W. ABBEEL, P. (2017). DOMAIN RANDOMIZATION FOR TRANSFERRING DEEP NEURAL NETWORKS FROM SIMULATION TO THE REAL WORLD. *2017 IEEE/RSJ INTERNATIONAL CONFERENCE ON INTELLIGENT ROBOTS AND SYSTEMS (IROS)*, PP 23-30. AVAILABLE ONLINE AT: https://ieeexplore.ieee.org/abstract/document/8202133 (ACCESSED: 03 JULY 2021)

UNITY TECHNOLOGIES (2005). UNITY. *UNITY TECHNOLOGIES*. AVAILABLE ONLINE AT: https://unity.com/ (ACCESSED: 15 NOVEMBER 2021)

VIDAKIS, N. BARIANOS, A. K. TRAMPAS, A. M. PAPADAKIS, S. KALOGIANNAKIS, M. VASSILAKIS, K. (2019). GENERATING EDUCATION IN-GAME DATA: THE CASE OF AN ANCIENT THEATRE SERIOUS GAME. *CSEDU 2019 - 11TH INTERNATIONAL CONFERENCE ON COMPUTER SUPPORTED EDUCATION*. AVAILABLE ONLINE AT: https://pdfs.semanticscholar.org/7f41/26cab6f8228e78f6b5d4ce361ee42e707f83.pdf (ACCESSED: 06 JUNE 2021)

VRAJITORU, D. (2018). GLOBAL TO LOCAL FOR PATH DECISION USING NEURAL NETWORKS. *PRAI 2018: PROCEEDINGS OF THE INTERNATIONAL CONFERENCE ON PATTERN RECOGNITION AND ARTIFICIAL INTELLIGENCE*, PP 117-123. AVAILABLE ONLINE AT: https://dl.acm.org/doi/10.1145/3243250.3243269 (ACCESSED: 20 OCTOBER 2021)

WANG, S. JIA, D. WENG, X. (2018). DEEP REINFORCEMENT LEARNING FOR AUTONOMOUS DRIVING. *CMU 10703 DEEP REINFORCEMENT LEARNING AND CONTROL COURSE PROJECT*. AVAILABLE ONLINE AT: https://arxiv.org/abs/1811.11329 (ACCESSED: 07 AUGUST 2021)

WARDHANA, M. I. (2021). LEARNING THROUGH A SOCIAL GAMING PLATFORM. *INTERNATIONAL CONFERENCE ON ART, DESIGN, EDUCATION AND CULTURAL STUDIES (ICADECS)*, PP 221-226 AVAILABLE ONLINE AT: https://knepublishing.com/index.php/KnE-Social/article/view/9199 (ACCESSED: 15 NOVEMBER 2021)

WILKINSON, P. (2016). A BRIEF HISTORY OF SERIOUS GAMES. *CHAPTER IN ENTERTAINMENT COMPUTING AND SERIOUS GAMES*, LNCS 9970, PP. 17-41, SPRINGER. AVAILABLE ONLINE AT: https://link.springer.com/chapter/10.1007/978-3-319-46152-6_2 (ACCESSED: 10 AUGUST 2021)

WONG, A. BÄCK, T. KONONOVA, A. V. PLAAT, A. (2021). MULTIAGENT DEEP REINFORCEMENT LEARNING: CHALLENGES AND DIRECTIONS TOWARDS HUMAN-LIKE APPROACHES. *ARTIFICIAL INTELLIGENCE REVIEW*. AVAILABLE ONLINE AT: https://arxiv.org/abs/2106.15691 (ACCESSED: 12 JULY 2021)

WYMANN, B. (2001). TORCS - THE OPEN RACING CAR SIMULATOR. *SOURCEFORGE*. AVAILABLE ONLINE AT: https://sourceforge.net/projects/torcs/ (ACCESSED: 04 JULY 2021)

YANNAKAKIS, G. N. TOGELIUS, J. (2018). ARTIFICIAL INTELLIGENCE AND GAMES. *SPRINGER*. AVAILABLE ONLINE AT: http://gameaibook.org/book.pdf (ACCESSED: 01 JULY 2021)

View publication stats